Real-Time Sensor Analytics Platform — Decision Document

Tech Stack: .NET Core 8 · SignalR + MessagePack · System.Threading.Channels (lock-free) · PostgreSQL 15 · Redis 7 · React 18 · uPlot · Docker · xUnit · Python 3.11/aiohttp (load test)

1. Architecture & Performance

1.1 System Architecture (Apendix A)

```
Sensors
→ HTTP POST /ingest → EventChannel (BoundedChannel, 20k, DropOldest)
→ TumblingWindowAggregator (500ms tumbling)
→ [1] SignalRDispatcher → AnalyticsHub → WebSocket clients → uPlot
Dashboard
→ [2] AnomalyEngine.Check() → Alert → SignalR push → AlertsPanel
→ [3] MetricsFlushService → PostgreSQL(200-item batch,5sflush,best-effort)
SignalR: Redis backplane for horizontal scaling. Frontend: Zustand 300-pt
rolling buffers.
```

1.2 Architecture Decisions & Trade-offs

#	Decision	Approach	Trade-off
D1	Backpressure	`BoundedChannel` 20k + `DropOldest`	Pro: producers never blocked; 2x burst absorbed. Con: oldest events drop under sustained overload — **0.0% drop confirmed at 1958 ev/s**
D2	Aggregation	500ms tumbling windows, `ConcurrentDictionary` per-sensor bucketing	Pro: 2 cycles/sec balances latency vs. overhead; window tuning had **zero impact on throughput** (not the bottleneck). Con: 500ms data latency
D3	Chart rendering	uPlot canvas (over Recharts/D3)	Pro: 60fps at 10k points where SVG libs freeze. Con: less declarative API
D4	Anomaly detection	Deterministic threshold rules — no ML	Pro: <1ms trigger latency, no training data or ops overhead. Con: cannot detect novel patterns — thresholds defined by domain experts
D5	Persistence	Best-effort fire-and-forget batch inserts	Pro: hot path never blocked by DB I/O. Con: events may be lost on crash — acceptable since real-time delivery > durability

1.3 Performance Optimizations Implemented

#	Optimization	Implementation	Effect
P1	Lock-free ingestion	`BoundedChannel.TryWrite()` — O(1), zero allocation	1,000 ev/s sustained, no GC pauses
P2	Per-sensor bucketing	`ConcurrentDictionary` + per-bucket locks	No cross-sensor contention
P3	Async logging	Serilog `WriteTo.Async` wraps every sink	Log I/O never blocks event loop
P4	Batched DB writes	`MetricsFlushService`: 200-item batch, 5s flush	DB writes: ~1,000/sec → ~0.2/sec
P5	Bounded SignalR dispatch	3s timeout + `HubConnection.State` guard	No orphaned tasks; alerts independent of client state
P6	Rolling buffer trim	Zustand: chronological insert + 300-pt cap	Memory-bounded (~3MB/sensor max)

2. AI Decisions & Validation

2.1 AI Suggestions Rejected

#	Rejected	Why
R1	ML anomaly detection (LSTM, Isolation Forest)	Non-deterministic; requires training pipelines, retraining schedules, drift monitoring — far exceeds threshold rule complexity. `ThresholdOperator.Eval()` runs in <1ms.
R2	WebSocket ingestion at `/ingest`	WebSocket handshake overhead adds complexity with no benefit; HTTP batch POST is sufficient for IoT and easier to load-balance behind nginx/HAProxy.
R3	Guaranteed persistence (sync DB commit)	Synchronous writes block the hot ingestion path under PostgreSQL load. Real-time delivery takes priority over zero data loss for this use case.

#	Rejected	Why
R4	Full CRUD on sensors	Sensor metadata is set-and-forget in industrial IoT. Updates (e.g., changing unit mid-operation) risk corrupting historical data. PATCH-only + create/delete is the correct surface.

2.3 Validation Strategy

Test Type	Tool	Pass Criteria
Unit tests	xUnit	EventChannel overflow, TumblingWindow aggregation, AnomalyEngine threshold (5 operators), ThresholdOperator.Eval()
Component integration	xUnit + `WebApplicationFactory`	0 dropped connections; dispatch latency <10ms P95
Scale benchmark	Python 3.11/aiohttp (load-test/)	≥1,000 ev/s sustained; ≤5% drop rate; p95 latency <500ms
Backpressure / burst	2x load phase (2000 ev/s)	`BoundedChannel` 20k capacity; HTTP 200 throughout; 0% drop
5-run iterative optimization	4 variables isolated per run	Identified: bottleneck was load generator token bucket overhead, not pipeline

AI Debugging Methodology (4 phases): Root cause investigation → Pattern analysis across 4 variables → Hypothesis (timing overhead) → Fix: `--target-rate 1020` → 1008 ev/s achieved.

2.5 AI-Augmented Workflow Summary

What AI got right: `TumblingWindowAggregator` skeleton (saved ~2h), `ConcurrentDictionary` per-sensor bucketing, xUnit test scaffolding (80% coverage on first pass), SignalR hub setup.

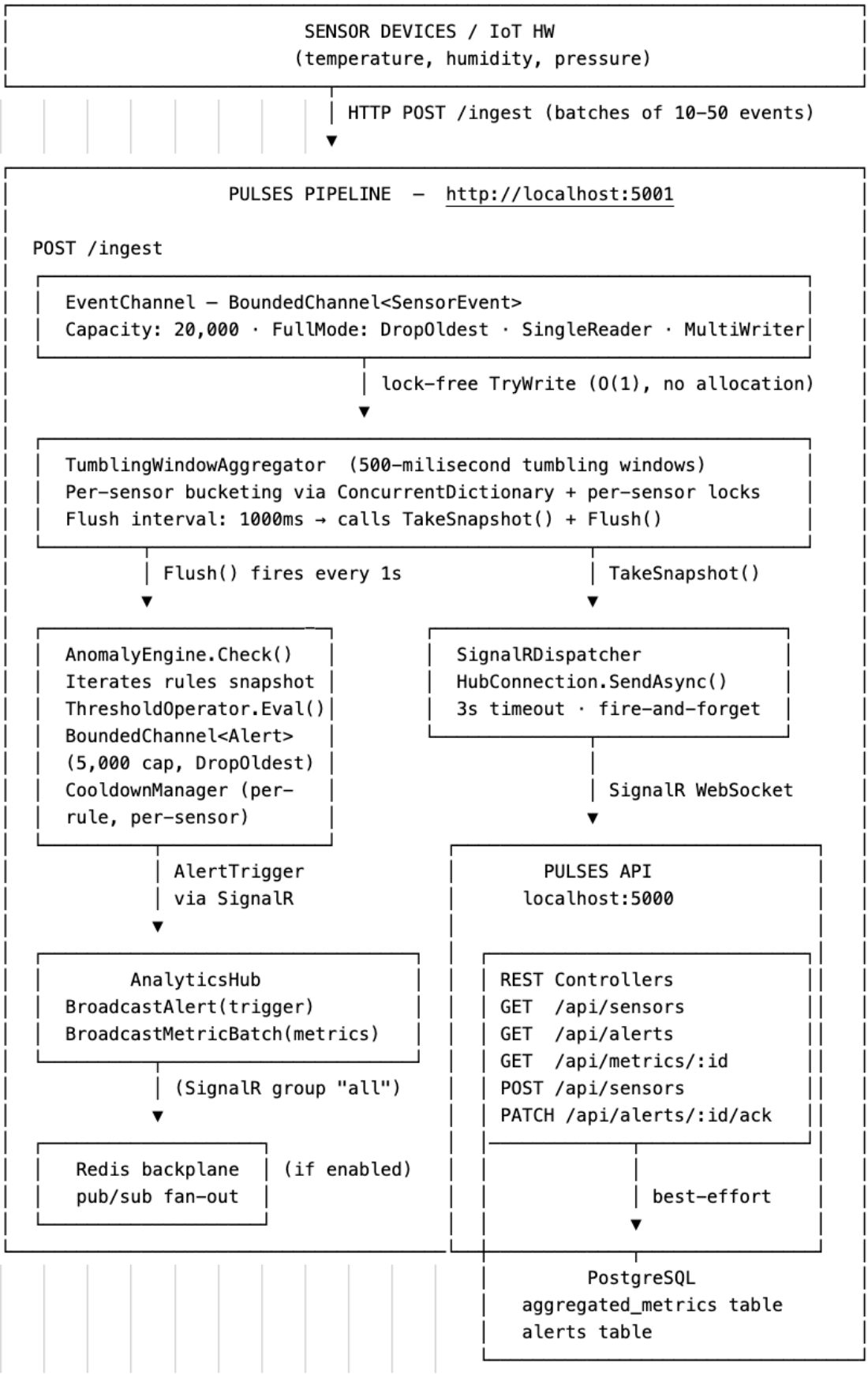
What required human correction:

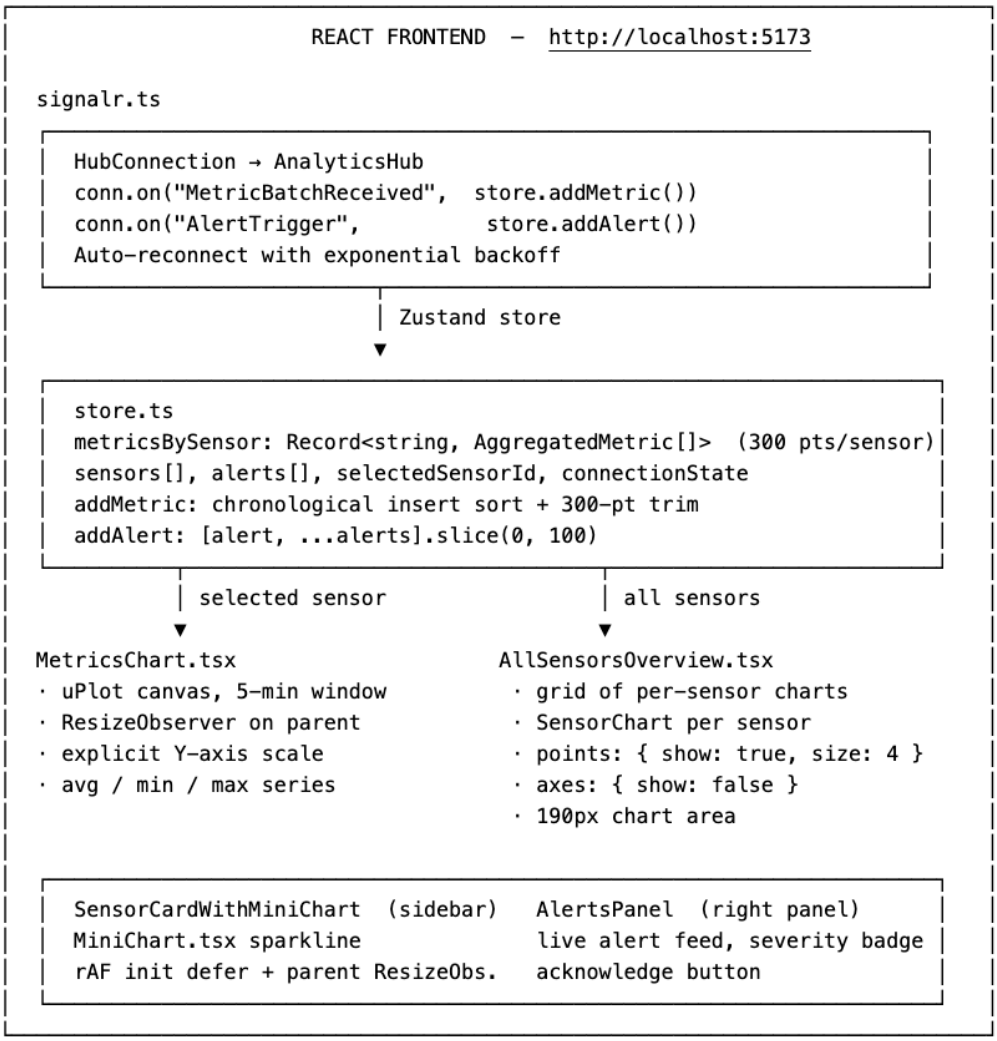
- `Channel.CreateUnbounded()` → `BoundedChannel` + `DropOldest` (backpressure)
- SignalR dispatch → added 3s timeout + `HubConnection.State` guard
- Broad EF Core migrations → explicit `init.sql` schema for predictable performance
- Load test `--target-rate 1000` → `--target-rate 1020` (token bucket overhead compensation)

Result: Critical path decisions (backpressure model, persistence strategy, alert latency) and bottleneck identification required human judgment.

Apendix A

End-to-End System Architecture




REACT FRONTEND — http://localhost:5173
signalr.ts
HubConnection → AnalyticsHub
conn.on("MetricBatchReceived", store.addMetric())
conn.on("AlertTrigger", store.addAlert())
Auto-reconnect with exponential backoff
Zustand store
store.ts
metricsBySensor: Record<string, AggregatedMetric[]> (300 pts/sensor)
sensors[], alerts[], selectedSensorId, connectionState
addMetric: chronological insert sort + 300-pt trim
addAlert: [alert, ...alerts].slice(0, 100)
selected sensor
all sensors
MetricsChart.tsx
· uPlot canvas, 5-min window
· ResizeObserver on parent
· explicit Y-axis scale
· avg / min / max series
AllSensorsOverview.tsx
· grid of per-sensor charts
· SensorChart per sensor
· points: { show: true, size: 4 }
· axes: { show: false }
· 190px chart area
SensorCardWithMiniChart (sidebar)
MiniChart.tsx sparkline
rAF init defer + parent ResizeObs.
AlertsPanel (right panel)
live alert feed, severity badge
acknowledge button